

AI-Powered Transaction Processing Pipeline

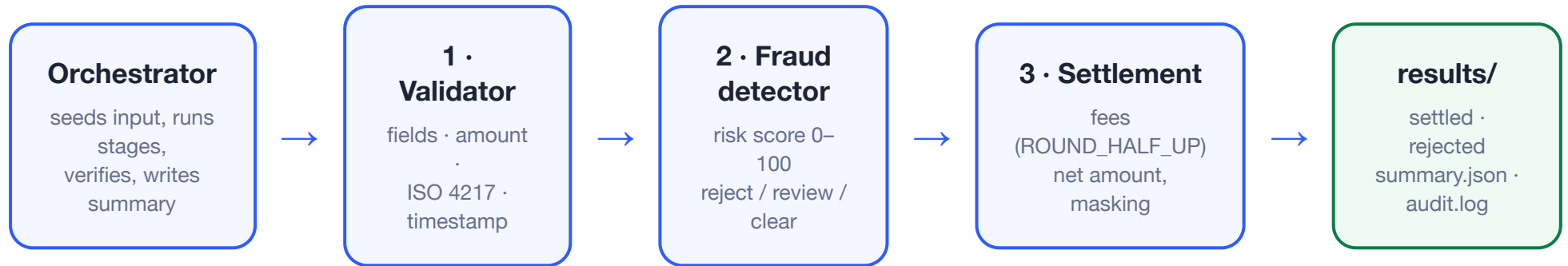
GenAI & Agentic AI for Software Engineering — Homework 6 · Final Capstone

Maksym Chykalo · August 2026

Node.js · decimal.js · React · node:test · FastMCP · Claude Code (agents, skills, hooks, MCP)

Architecture — a file-based pipeline

Stages communicate **only** through JSON message envelopes in shared directories (shared/input → processing → output → results). Each stage claims the envelopes addressed to it, works in processing/, and emits a fresh envelope — written atomically.



Money & PII guardrails

- Amounts are decimal.js Decimals — native floats are forbidden everywhere
- Accounts persist masked: ACC-1001 → ACC-**01; descriptions never logged

Observability

- ISO 8601 audit line per operation: timestamp | stage | txn | outcome
- React dashboard, CLI table, and an MCP server over the same results

Pipeline stages & business rules

Validation → reject codes

Check	Code
7 required fields present	MISSING_FIELD
positive decimal, ≤ 2 dp	INVALID_AMOUNT
ISO 4217 allowlist	INVALID_CURRENCY
ISO 8601 timestamp	INVALID_TIMESTAMP

Settlement fees

Type	Fee
wire_transfer	0.10%, min 25.00
transfer / refund / other	0.25 / 0.00 / 0.50

Fraud scoring (additive, cap 100)

Rule	Condition	+
VERY_HIGH_VALUE	> 50,000	50
HIGH_VALUE	10,000 – 50,000	40
NEAR_THRESHOLD	9,000 – 9,999.99 (structuring)	30
WATCHLIST_ACCOUNT	destination watchlisted	40
OFF_HOURS	00:00–04:59 UTC	20
CROSS_BORDER	country ≠ US	15

≥ 70 rejected

30–69 review

< 30 cleared

The four-agent AI workflow (Claude Code)

Agent	Deliverable	Workflow artifact
1 · Specification	specification.md — 5 sections, task-per-stage, testable outcome table	spec-writer agent + /write-spec skill
2 · Code generation	pipeline, orchestrator, React dashboard	context7 MCP : 3 documented queries (decimal.js, Express, Vite) in research-notes.md
3 · Unit tests	63 tests, ~93% line coverage	unit-test-writer agent + coverage-gate hook : push blocked < 80%
4 · Documentation	README (author line), HOWTORUN, this deck	screenshots + evidence captured reproducibly into docs/

Skills

- /run-pipeline — full run + result summary + rejection report
- /validate-transactions — validator dry-run table, no processing

Hooks & MCP

- PreToolUse hook + native git pre-push — both call one coverage gate script
- Custom FastMCP server: get_transaction_status, list_pipeline_results, pipeline://summary

Demo — all 8 sample transactions accounted for

Transaction Processing Pipeline

validator → fraud detector → settlement

Run pipeline

8Transactions

5Settled

3Needs review

3Rejected

Settled net totals: 104599.50 USD · 499.75 EUR — run 6c511987 finished 2026-08-18T09:05:26.489Z

Transaction	From → To	Amount	Risk	Status	Reason
TXN001	ACC-**01 → ACC-**01	1500.00 USD	0	Settled	
TXN002	ACC-**02 → ACC-**01	25000.00 USD	40	Settled - review	HIGH_VALUE
TXN003	ACC-**03 → ACC-**99	9999.99 USD	70	Rejected	FRAUD_SUSPECTED NEAR_THRESHOLD, WATCHLIST_ACCOUNT
TXN004	ACC-**04 → ACC-**00	500.00 EUR	35	Settled - review	OFF_HOURS, CROSS_BORDER
TXN005	ACC-**05 → ACC-**00	75000.00 USD	50	Settled - review	VERY_HIGH_VALUE
TXN006	ACC-**06 → ACC-**00	200.00 XYZ	—	Rejected	INVALID_CURRENCY
TXN007	ACC-**07 → ACC-**00	-100.00 GBP	—	Rejected	INVALID_AMOUNT
TXN008	ACC-**08 → ACC-**00	3200.00 USD	0	Settled	

Homework 6 capstone — created by Maksym Chykalo

5settled

3need review

3rejected

- **TXN003** — FRAUD_SUSPECTED (risk 70: structuring \$9,999.99 + watchlisted ACC-9999)
- **TXN006** — INVALID_CURRENCY (XYZ)
- **TXN007** — INVALID_AMOUNT (-100.00)
- Settled net totals: **104,599.50 USD · 499.75 EUR**

Testing & quality gates

63

tests (node:test)

92.6%

line coverage (c8)

80%

hard push gate

0

floats holding money

What the suite proves

- Unit tests per stage: every rejection code, every fraud rule in isolation
- Boundaries: 0.01 amounts, 3-dp rejection, 10,000.00 / 50,000.00 / 04:59, wire-fee minimum at 25,000, score cap 100
- Integration: full pipeline in a temp root — statuses match the spec's outcome table byte-for-byte
- PII invariants: no unmasked account appears in results or audit log

Coverage gate in practice

- One gate script → two enforcement points: Claude Code PreToolUse hook (exit 2 blocks the tool call) and native `.git/hooks/pre-push`
- Fails closed: no tests = 0% = blocked
- Observed live: the hook intercepted a `git push` during development and blocked it until the suite landed

Lessons learned

- **Spec first pays off.** The expected-outcomes table written in Task 1 became the integration-test oracle in Task 5 — the pipeline matched it on the first run.
- **Design the sample data into the rules.** Every fraud weight was tuned so the 8 sample records exercise every decision branch (a rejection exactly at threshold 70, flags at 35/40/50).
- **Changing stacks is cheap when the spec is the contract.** Switching Python → Node.js + React was a one-document revision, then regeneration.
- **Hooks beat discipline.** A coverage gate that fails closed caught its first push the same day it was written.
- **MCP is two-way.** context7 fed docs into code generation; the custom FastMCP server exposes the pipeline back to the AI — the workflow both consumes and produces MCP.
- **Environment quirks are real work:** corporate npm registry, Node 18 vs 20 (undici needs the `File` global), sandboxed networking — all documented in HOWTORUN/research notes.